AGLC Law Firm CRM: Knowledge Transfer & Operations Guide

Document Version: 1.0
Date: March 11, 2026
Project: AGLC Law Firm CRM (PPP Platform)
Audience: Development Team, Operations Team, System Administrators, Future Maintainers

---

Table of Contents

1. System Overview
2. Architecture & Design
3. Technology Stack
4. Database Schema
5. API & Integration
6. AI Agent System
7. Development Workflow
8. Deployment & Operations
9. Troubleshooting Guide
10. Maintenance & Support
11. Security & Compliance
12. Scaling & Performance

---

1. System Overview

1.1 What is the AGLC CRM?

The AGLC Law Firm CRM is a comprehensive practice management system designed to streamline case management, task tracking, client relationships, and team collaboration. It combines traditional CRM functionality with advanced AI agents to automate routine tasks and provide intelligent insights.

1.2 Core Value Propositions

For Lawyers & Staff:

- Centralized case and task management
- Automated routine tasks (data entry, document tagging)
- Intelligent task assignment based on skills
- Real-time alerts and deadline tracking
- Gamification and recognition system
- Wellness support and work-life balance features

For Clients:

- VIP portal with case visibility
- Automated status updates
- Direct communication channels
- Document access and sharing
- Satisfaction tracking

For Leadership:

- Executive dashboard with full visibility
- Performance analytics and insights
- Risk monitoring and alerts
- Financial metrics and profitability tracking
- Strategic planning tools (SWOT analysis, optimization recommendations)

1.3 Key Features (MVP Phase 1)

Feature	Purpose	Status
Case Management	Track cases through phases	✅ Phase 1
Task Management	Assign and track tasks	✅ Phase 1
VIP Client Portal	Client visibility and communication	✅ Phase 1
Gamification	Awards and recognition system	✅ Phase 1
Executive Dashboard	Leadership visibility	✅ Phase 1
Planner Agent	AI-generated case roadmaps	✅ Phase 1
Tracker Agent	Deadline monitoring and alerts	✅ Phase 1
Wellness Features	Break reminders, prayer times	✅ Phase 1

---

2. Architecture & Design

2.1 System Architecture Overview

```
┌──────────────────────────────────────────────────────────────────┐
│                    AGLC CRM SYSTEM ARCHITECTURE                  │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │          FRONTEND (React 19 + Tailwind CSS)              │   │
│  │  ├─ Dashboard & Navigation                               │   │
│  │  ├─ Case Management UI                                  │   │
│  │  ├─ Task Board (Kanban)                                 │   │
│  │  ├─ Client Portal                                       │   │
│  │  ├─ Executive Dashboard                                │   │
│  │  └─ User Settings & Profile                            │   │
│  └──────────────────────────────────────────────────────────┘   │
│                            ▲                                     │
│                            │ (tRPC over HTTP)                   │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │          BACKEND (Express 4 + Node.js)                   │   │
│  │  ├─ tRPC Router (Type-safe RPC)                         │   │
│  │  ├─ Authentication & Authorization                     │   │
│  │  ├─ Business Logic & Procedures                        │   │
│  │  ├─ AI Agent Orchestration                            │   │
│  │  ├─ Notification System                               │   │
│  │  └─ Integration Handlers                              │   │
│  └──────────────────────────────────────────────────────────┘   │
│                            ▲                                     │
│                            │ (SQL Queries)                      │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │          DATABASE (PostgreSQL)                           │   │
│  │  ├─ Users & Authentication                             │   │
│  │  ├─ Clients & Companies                                │   │
│  │  ├─ Cases & Phases                                     │   │
│  │  ├─ Tasks & Assignments                                │   │
│  │  ├─ Communications & History                           │   │
│  │  ├─ Performance & Analytics                            │   │
│  │  └─ Audit Logs                                         │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │          EXTERNAL SERVICES                               │   │
│  │  ├─ OpenAI GPT-4 (LLM)                                 │   │
```

```
│  │  ├─ Twilio (WhatsApp, SMS)                          │   │
│  │  ├─ SendGrid (Email)                               │   │
│  │  ├─ AWS S3 (File Storage)                          │   │
│  │  └─ Google Maps (Location Services)                │   │
│  └──────────────────────────────────────────────────┘   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

2.2 Data Flow Architecture

Case Creation Flow:

```
Client Request
    ↓
Create Case (Frontend)
    ↓
Save to Database
    ↓
Trigger Planner Agent
    ↓
Generate Task Roadmap
    ↓
Create Tasks in Database
    ↓
Notify Team
    ↓
Display on Dashboard
```

Task Execution Flow:

```
Task Created
    ↓
Assign to Team Member
    ↓
Team Member Starts Task
    ↓
Tracker Agent Monitors Progress
    ↓
Task Completed
    ↓
Executor Agent Processes Output
    ↓
Crisis Manager Validates Quality
    ↓
Update Case Status
    ↓
Notify Client & Team
```

2.3 Design Principles

1. Type Safety First

- Use TypeScript throughout (frontend and backend)
- Use Zod for runtime validation
- Use tRPC for end-to-end type safety
- Never use `any` type

2. Separation of Concerns

- Frontend: UI and user interactions
- Backend: Business logic and data access
- Database: Data persistence
- External Services: Specialized functions

3. Scalability

- Stateless backend (can run multiple instances)
- Database connection pooling

- Caching layer (Redis) for frequently accessed data
- Async job processing (Bull.js) for long-running tasks

4. Security

- Encrypt sensitive data at rest and in transit
- Implement role-based access control (RBAC)
- Audit all access and changes
- Never log sensitive data

5. Maintainability

- Clear code organization and naming
- Comprehensive documentation
- Automated testing (unit, integration, e2e)
- Continuous integration/deployment (CI/CD)

---

3. Technology Stack

3.1 Frontend Stack

Technology	Version	Purpose	Why Chosen
React	19	UI Framework	Modern, component-based, large ecosystem
TypeScript	5.9	Type Safety	Catch errors at compile time
Tailwind CSS	4	Styling	Utility-first, responsive, customizable
shadcn/ui	Latest	UI Components	Pre-built, accessible, customizable
TanStack Query	5.90	State Management	Server state, caching, sync
React Hook Form	7.64	Form Handling	Type-safe, performant, minimal re-renders
Zod	4.1	Validation	Runtime type checking
Recharts	2.15	Charting	React-native, interactive, responsive
Wouter	3.3	Routing	Lightweight, simple, fast
Sonner	2.0	Notifications	Toast notifications, beautiful

3.2 Backend Stack

Technology	Version	Purpose	Why Chosen
Node.js	22	Runtime	JavaScript on server, unified language
Express	4	Web Framework	Lightweight, flexible, mature
tRPC	11	API Layer	Type-safe RPC, end-to-end type safety
TypeScript	5.9	Type Safety	Catch errors at compile time
PostgreSQL	14+	Database	Relational, ACID, JSON support
Drizzle ORM	0.44	Database Access	Type-safe, lightweight, SQL-first
Bull.js	Latest	Job Queue	Redis-backed, reliable, scalable
OpenAI SDK	Latest	LLM Integration	State-of-the-art, reliable
Twilio SDK	Latest	WhatsApp/SMS	Reliable, global coverage
SendGrid SDK	Latest	Email	Scalable, reliable, good deliverability

3.3 Infrastructure Stack

Component	Technology	Purpose
Hosting	Manus Platform	Managed, scalable, no DevOps overhead
Database	PostgreSQL (Managed)	Automated backups, high availability
File Storage	AWS S3	Scalable, secure, cost-effective
Caching	Redis	Session storage, data caching
Monitoring	Manus Logs + Sentry	Error tracking, performance monitoring
CI/CD	GitHub Actions	Automated testing, deployment
Version Control	GitHub	Code management, collaboration

3.4 Development Tools

Tool	Purpose	Usage
Vite	Build Tool	Fast development, optimized production builds
Vitest	Testing Framework	Unit and integration tests
Prettier	Code Formatter	Consistent code style
ESLint	Linting	Code quality and error detection
Git	Version Control	Code management and history
GitHub	Repository	Code hosting and collaboration
Figma	Design	UI/UX design and prototyping
Postman	API Testing	Manual API testing and debugging

---

4. Database Schema

4.1 Core Tables

Users Table

```sql
CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  openId VARCHAR(64) UNIQUE NOT NULL,
  name TEXT,
  email VARCHAR(320),
  loginMethod VARCHAR(64),
  role ENUM('user', 'admin') DEFAULT 'user',
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  lastSignedIn TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Purpose: Store user authentication and profile information
Key Fields:

- `openId` : Manus OAuth identifier (unique per user)
- `role` : Determines access level (user or admin)
- `lastSignedIn` : Track user activity

Clients Table

```sql
CREATE TABLE clients (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(320),
  phone VARCHAR(20),
  company VARCHAR(255),
  vipStatus BOOLEAN DEFAULT FALSE,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

Purpose: Store client information
Key Fields:

- `vipStatus` : Indicates high-value clients with special treatment
- `company` : Client's organization

Cases Table

```sql
CREATE TABLE cases (
  id INT PRIMARY KEY AUTO_INCREMENT,
  clientId INT NOT NULL,
  caseType ENUM('Corporate', 'RealEstate', 'Litigation', 'IP', 'Other'),
  status ENUM('Intake', 'Active', 'OnHold', 'Completed', 'Closed') DEFAULT
'Intake',
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  deadline DATE,
  assignedTeam JSON,
  metadata JSON,
  FOREIGN KEY (clientId) REFERENCES clients(id)
);
```

Purpose: Store case information and track progress
Key Fields:

- `caseType`: Determines which template to use
- `status`: Current phase of the case
- `assignedTeam`: JSON array of user IDs assigned to case
- `metadata`: Custom fields per case type

Phases Table

```sql
CREATE TABLE phases (
  id INT PRIMARY KEY AUTO_INCREMENT,
  caseId INT NOT NULL,
  phaseNumber INT,
  phaseName VARCHAR(255),
  status ENUM('Pending', 'InProgress', 'Completed') DEFAULT 'Pending',
  startDate DATE,
  endDate DATE,
  teamRequirements JSON,
  FOREIGN KEY (caseId) REFERENCES cases(id)
);
```

Purpose: Break cases into phases with specific requirements
Key Fields:

- `phaseNumber`: Order of phases (1, 2, 3, etc.)
- `teamRequirements`: JSON specifying roles needed

Tasks Table

```sql
CREATE TABLE tasks (
  id INT PRIMARY KEY AUTO_INCREMENT,
  phaseId INT NOT NULL,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  assignedTo INT,
  status ENUM('Pending', 'InProgress', 'Blocked', 'Completed') DEFAULT
'Pending',
  priority ENUM('Low', 'Medium', 'High', 'Critical') DEFAULT 'Medium',
  deadline TIMESTAMP,
  estimatedHours DECIMAL(5,2),
  actualHours DECIMAL(5,2),
  dependencies JSON,
  acceptanceCriteria TEXT,
  FOREIGN KEY (phaseId) REFERENCES phases(id),
  FOREIGN KEY (assignedTo) REFERENCES users(id)
);
```

Purpose: Store individual tasks within phases
Key Fields:

- `priority`: Determines alert frequency and importance
- `dependencies`: JSON array of task IDs that must complete first
- `estimatedHours` vs `actualHours`: Track efficiency

Communications Table

```sql
CREATE TABLE communications (
  id INT PRIMARY KEY AUTO_INCREMENT,
  clientId INT NOT NULL,
  caseId INT,
  type ENUM('Email', 'Call', 'Meeting', 'Message', 'Document'),
  direction ENUM('Inbound', 'Outbound'),
  content TEXT,
  senderId INT,
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  sentiment ENUM('Positive', 'Neutral', 'Negative'),
  FOREIGN KEY (clientId) REFERENCES clients(id),
  FOREIGN KEY (caseId) REFERENCES cases(id),
  FOREIGN KEY (senderId) REFERENCES users(id)
);
```

Purpose: Centralized log of all client interactions
Key Fields:

- `sentiment`: AI-analyzed sentiment of communication
- `type`: Different communication channels

Performance Table

```sql
CREATE TABLE performance (
  id INT PRIMARY KEY AUTO_INCREMENT,
  userId INT NOT NULL,
  period DATE,
  tasksCompleted INT,
  qualityScore DECIMAL(3,2),
  clientSatisfactionAvg DECIMAL(2,1),
  pointsEarned INT,
  burnoutRiskScore DECIMAL(3,2),
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (userId) REFERENCES users(id)
);
```

Purpose: Track individual performance metrics
Key Fields:

- `qualityScore`: 1-100 scale
- `burnoutRiskScore`: Identifies overworked team members
- `period`: Monthly tracking

4.2 Relationships Diagram

```
Users (1) ──────────────────────── (M) Tasks
  │                                   │
  │                                   │
  ├─ (M) Cases                        ├─ (M) Phases
  │                                   │
  ├─ (M) Communications               └─ (M) Cases
  │                                       │
  ├─ (M) Performance                      ├─ (M) Clients
  │                                       │
  └─ (M) Audit Logs                       └─ (M) Communications
```

4.3 Indexing Strategy

High-Priority Indexes:

```sql
CREATE INDEX idx_users_openId ON users(openId);
CREATE INDEX idx_cases_clientId ON cases(clientId);
CREATE INDEX idx_cases_status ON cases(status);
CREATE INDEX idx_tasks_assignedTo ON tasks(assignedTo);
CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_deadline ON tasks(deadline);
CREATE INDEX idx_communications_clientId ON communications(clientId);
CREATE INDEX idx_communications_timestamp ON communications(timestamp);
CREATE INDEX idx_performance_userId ON performance(userId);
```

Why: These queries are run frequently and need to be fast.

---

5. API & Integration

5.1 tRPC Router Structure

The backend uses tRPC for type-safe API calls. All procedures are defined in `server/routers.ts`.

Router Organization:

```
appRouter = {
  auth: {
    me: publicProcedure.query(),
    logout: publicProcedure.mutation(),
  },
  cases: {
    list: protectedProcedure.query(),
    create: protectedProcedure.mutation(),
    update: protectedProcedure.mutation(),
    getById: protectedProcedure.query(),
    delete: protectedProcedure.mutation(),
  },
  tasks: {
    list: protectedProcedure.query(),
    create: protectedProcedure.mutation(),
    update: protectedProcedure.mutation(),
    assign: protectedProcedure.mutation(),
    complete: protectedProcedure.mutation(),
  },
  clients: {
    list: protectedProcedure.query(),
    create: protectedProcedure.mutation(),
    getById: protectedProcedure.query(),
    updatePersona: protectedProcedure.mutation(),
  },
  dashboard: {
    getExecutiveOverview: adminProcedure.query(),
    getTeamPerformance: adminProcedure.query(),
    getFinancialMetrics: adminProcedure.query(),
  },
  ai: {
    generateCasePlan: protectedProcedure.mutation(),
    analyzeCaseRisk: adminProcedure.query(),
    suggestTaskAssignment: protectedProcedure.query(),
  },
  notifications: {
    list: protectedProcedure.query(),
    markAsRead: protectedProcedure.mutation(),
    subscribe: protectedProcedure.subscription(),
  },
}
```

5.2 Authentication Flow

Manus OAuth Integration:

1. User clicks “Login” on frontend
2. Frontend redirects to Manus OAuth portal with `state` parameter containing origin
3. User authenticates with Manus
4. Manus redirects to `/api/oauth/callback` with authorization code
5. Backend exchanges code for token
6. Backend creates/updates user in database
7. Backend sets session cookie
8. Frontend reads auth state with `trpc.auth.me.useQuery()`

Key Files:

- `server/_core/oauth.ts`: OAuth configuration
- `server/_core/context.ts`: Context building (includes user)
- `server/_core/trpc.ts`: Procedure definitions (public, protected, admin)

5.3 External Service Integrations

OpenAI LLM Integration

```
import { invokeLLM } from "./server/_core/llm";

const response = await invokeLLM({
  messages: [
    { role: "system", content: "You are a legal case planner." },
    { role: "user", content: "Create a plan for a corporate litigation
case." },
  ],
});
```

Use Cases:

- Planner Agent: Generate case roadmaps
- Executor Agent: Extract information from documents
- Consultant Agent: Analyze workflows

Twilio WhatsApp Integration

```
import twilio from 'twilio';

const client = twilio(accountSid, authToken);
const message = await client.messages.create({
  body: 'Your case status update...',
  from: 'whatsapp:+1234567890',
  to: 'whatsapp:+9966xxxxxxxx',
});
```

Use Cases:

- Send case updates to clients
- Receive client messages
- Two-way communication

SendGrid Email Integration

```
import sgMail from '@sendgrid/mail';

await sgMail.send({
  to: 'client@example.com',
  from: 'noreply@aglc.com',
  subject: 'Case Update',
  html: '<p>Your case has progressed to the next phase...</p>',
});
```

Use Cases:

- Send case updates
- Send task assignments
- Send performance reports

AWS S3 File Storage

```
import { storagePut } from "./server/storage";

const { url } = await storagePut(
  `cases/${caseId}/documents/${fileName}`,
  fileBuffer,
  'application/pdf'
);
```

Use Cases:

- Store case documents
- Store client files
- Store generated reports

---

6. AI Agent System

6.1 AI Agent Architecture

Agent Orchestration Engine:

```
┌──────────────────────────────────────────────────────┐
│      AI Agent Orchestration Engine                   │
│  (Central coordinator for all AI agents)             │
├──────────────────────────────────────────────────────┤
│                                                      │
│  ┌──────────────┐  ┌──────────────┐                 │
│  │ Planner      │  │ Tracker      │                 │
│  │ Agent        │  │ Agent        │                 │
│  └──────────────┘  └──────────────┘                 │
│                                                      │
│  ┌──────────────┐  ┌──────────────┐                 │
│  │ Executor     │  │ Crisis       │                 │
│  │ Agent        │  │ Manager      │                 │
│  └──────────────┘  └──────────────┘                 │
│                                                      │
│  ┌──────────────┐  ┌──────────────┐                 │
│  │ Resource/    │  │ Personality  │                 │
│  │ Knowledge    │  │ Agent        │                 │
│  └──────────────┘  └──────────────┘                 │
│                                                      │
│  ┌──────────────────────────────┐                   │
│  │ Consultant Agent             │                   │
│  └──────────────────────────────┘                   │
│                                                      │
└──────────────────────────────────────────────────────┘
```

6.2 Agent Specifications

Planner Agent (Phase 1)

Purpose: Generate case roadmaps and task sequences
Input: Case type, client requirements, deadline
Output: Structured task list with phases and dependencies
Implementation:

```
async function plannerAgent(caseData: CaseInput) {
  const prompt = `
    You are a legal case planner. Create a detailed task roadmap for:
    - Case Type: ${caseData.caseType}
    - Client Requirements: ${caseData.requirements}
    - Deadline: ${caseData.deadline}

    Return a JSON object with:
    {
      phases: [
        {
          name: string,
          tasks: [
            {
              title: string,
              description: string,
              estimatedHours: number,
              dependencies: string[]
            }
          ]
        }
      ]
    }
  `;

  const response = await invokeLLM({
    messages: [{ role: "user", content: prompt }],
    response_format: { type: "json_schema", ... }
  });

  return JSON.parse(response.choices[0].message.content);
}
```

Fallback: If LLM fails, use predefined templates

Tracker Agent (Phase 1)

Purpose: Monitor deadlines and send context-aware alerts
Input: Task list, current progress, team availability
Output: Alert notifications with urgency level
Implementation:

```
async function trackerAgent() {
  // Run every 15 minutes
  const urgentTasks = await db.query(`
    SELECT * FROM tasks
    WHERE status != 'Completed'
    AND deadline < NOW() + INTERVAL 3 HOUR
  `);

  for (const task of urgentTasks) {
    const urgency = calculateUrgency(task);
    await sendAlert(task.assignedTo, {
      title: `Urgent: ${task.title}`,
      message: `${task.title} is due in ${timeRemaining(task.deadline)}`,
      urgency: urgency,
    });
  }
}

function calculateUrgency(task) {
  const hoursRemaining = (task.deadline - now()) / 3600000;
  if (hoursRemaining < 1) return 'CRITICAL';
  if (hoursRemaining < 3) return 'HIGH';
  if (hoursRemaining < 24) return 'MEDIUM';
  return 'LOW';
}
```

Scheduling: Run every 15 minutes via Bull.js job queue

Executor Agent (Phase 2)

Purpose: Automate routine data entry and document processing
Input: Raw document data, structured templates
Output: Extracted and categorized information
Implementation:

```
async function executorAgent(document: DocumentData) {
  const prompt = `
    Extract the following information from this legal document:
    - Party names
    - Key dates
    - Obligations
    - Payment terms

    Return as JSON.
  `;

  const extracted = await invokeLLM({
    messages: [
      { role: "user", content: prompt },
      { type: "file_url", file_url: { url: document.url } }
    ],
    response_format: { type: "json_schema", ... }
  });

  await db.insert('documentData').values({
    documentId: document.id,
    extractedData: extracted,
    processedAt: new Date(),
  });
}
```

Scheduling: Triggered when document is uploaded

Crisis Manager (Phase 2)

Purpose: Identify high-risk scenarios and escalate
Input: Case data, task progress, client feedback
Output: Risk alerts with recommended actions
Implementation:

```
async function crisisManager(caseId: number) {
  const riskFactors = [];

  // Check for missed deadlines
  const overdueTasks = await db.query(`
    SELECT COUNT(*) as count FROM tasks
    WHERE caseId = ? AND deadline < NOW() AND status != 'Completed'
  `);
  if (overdueTasks.count > 0) {
    riskFactors.push({
      type: 'OVERDUE_TASKS',
      severity: 'HIGH',
      count: overdueTasks.count,
    });
  }

  // Check for negative client feedback
  const negativeFeedback = await db.query(`
    SELECT COUNT(*) as count FROM communications
    WHERE caseId = ? AND sentiment = 'Negative'
  `);
  if (negativeFeedback.count > 2) {
    riskFactors.push({
      type: 'NEGATIVE_FEEDBACK',
      severity: 'CRITICAL',
      count: negativeFeedback.count,
    });
  }

  if (riskFactors.length > 0) {
    await escalateToManagement(caseId, riskFactors);
  }
}
```

Scheduling: Run every 6 hours via Bull.js

6.3 Agent Monitoring & Logging

Log All Agent Actions:

```typescript
interface AgentLog {
  agentName: string;
  action: string;
  input: any;
  output: any;
  status: 'SUCCESS' | 'FAILURE';
  error?: string;
  executionTime: number;
  timestamp: Date;
}

async function logAgentAction(log: AgentLog) {
  await db.insert('agentLogs').values(log);
}
```

Monitor Agent Performance:

```typescript
async function getAgentMetrics(agentName: string) {
  return await db.query(`
    SELECT
      COUNT(*) as totalRuns,
      SUM(CASE WHEN status = 'SUCCESS' THEN 1 ELSE 0 END) as successCount,
      AVG(executionTime) as avgExecutionTime,
      MAX(executionTime) as maxExecutionTime
    FROM agentLogs
    WHERE agentName = ?
    AND timestamp > NOW() - INTERVAL 7 DAY
  `);
}
```

---

7. Development Workflow

7.1 Local Development Setup

Prerequisites:

- Node.js 22+
- PostgreSQL 14+

- Git
- VS Code (recommended)

Setup Steps:

1. **Clone Repository:**

```
git clone https://github.com/yourorg/aglc-crm.git
cd aglc-crm
```

2. **Install Dependencies:**

```
pnpm install
```

3. **Set Up Environment:**

```
cp .env.example .env.local
# Edit .env.local with your local database and API keys
```

4. **Create Database:**

```
pnpm db:push
```

5. **Start Development Server:**

```
pnpm dev
```

6. **Access Application:**

   - Frontend: http://localhost:5173
   - Backend: http://localhost:3000

7.2 Code Organization

Frontend Structure:

```
client/
├── src/
│   ├── pages/           # Page components
│   ├── components/      # Reusable UI components
│   ├── contexts/        # React contexts
│   ├── hooks/          # Custom hooks
│   ├── lib/            # Utilities (tRPC client, etc.)
│   ├── App.tsx         # Main app component
│   ├── main.tsx        # Entry point
│   └── index.css       # Global styles
└── public/             # Static assets (favicon, robots.txt only)
```

Backend Structure:

```
server/
├── routers.ts          # tRPC procedure definitions
├── db.ts               # Database query helpers
├── _core/              # Framework code (don't edit)
│   ├── index.ts        # Server entry point
│   ├── context.ts      # tRPC context
│   ├── trpc.ts         # Procedure definitions
│   ├── oauth.ts        # OAuth configuration
│   ├── llm.ts          # LLM integration
│   └── ...
└── agents/             # AI agent implementations
    ├── planner.ts
    ├── tracker.ts
    ├── executor.ts
    └── ...
```

7.3 Adding a New Feature

Step 1: Define Database Schema

```
// drizzle/schema.ts
export const newFeature = mysqlTable('newFeature', {
  id: int('id').autoincrement().primaryKey(),
  userId: int('userId').references(() => users.id),
  data: text('data'),
  createdAt: timestamp('createdAt').defaultNow(),
});
```

Step 2: Create Database Migration

```
pnpm db:push
```

Step 3: Add Query Helper

```
// server/db.ts
export async function getNewFeatureData(userId: number) {
  const db = await getDb();
  return db.select().from(newFeature).where(eq(newFeature.userId, userId));
}
```

Step 4: Add tRPC Procedure

```
// server/routers.ts
newFeature: router({
  list: protectedProcedure.query(({ ctx }) =>
    db.getNewFeatureData(ctx.user.id)
  ),
  create: protectedProcedure
    .input(z.object({ data: z.string() }))
    .mutation(({ ctx, input }) =>
      db.insert(newFeature).values({
        userId: ctx.user.id,
        data: input.data,
      })
    ),
}),
```

Step 5: Add Frontend UI

```
// client/src/pages/NewFeature.tsx
export default function NewFeature() {
  const { data, isLoading } = trpc.newFeature.list.useQuery();
  const createMutation = trpc.newFeature.create.useMutation();

  return (
    <div>
      {/* UI code */}
    </div>
  );
}
```

Step 6: Write Tests

```
// server/newFeature.test.ts
describe('newFeature', () => {
  it('should create new feature', async () => {
    const result = await caller.newFeature.create({ data: 'test' });
    expect(result).toBeDefined();
  });
});
```

Step 7: Run Tests

```
pnpm test
```

7.4 Git Workflow

Branch Naming:

- Feature: `feature/case-management`
- Bug fix: `bugfix/task-assignment-issue`
- Hotfix: `hotfix/critical-security-issue`

Commit Messages:

```
feat: add case management feature
fix: resolve task assignment bug
docs: update API documentation
test: add tests for case creation
chore: update dependencies
```

Pull Request Process:

1. Create feature branch from `main`
2. Make changes and commit
3. Push to GitHub
4. Create pull request with description
5. Request code review
6. Address feedback
7. Merge when approved

---

8. Deployment & Operations

8.1 Deployment Process

Manus Platform Deployment:

1. **Create Checkpoint:**

```
# Automatically done via Manus UI
```

2. **Run Tests:**

```
pnpm test
```

3. **Build:**

```
pnpm build
```

4. **Deploy:**
   - Click “Publish” button in Manus UI
   - Select checkpoint to deploy
   - Monitor deployment progress
5. **Verify:**
   - Check that application is running
   - Run smoke tests
   - Monitor error logs

8.2 Environment Variables

Development (.env.local):

```
DATABASE_URL=postgresql://user:password@localhost:5432/aglc_crm
OPENAI_API_KEY=sk-...
TWILIO_ACCOUNT_SID=AC...
TWILIO_AUTH_TOKEN=...
SENDGRID_API_KEY=SG...
```

Production (Set in Manus UI):

- All environment variables are managed in Manus UI
- Never commit `.env` files to Git
- Use `webdev_request_secrets` tool to add/update secrets

8.3 Monitoring & Logging

Log Files Location:

```
.manus-logs/
├── devserver.log          # Server startup, Vite HMR
├── browserConsole.log     # Client-side console output
├── networkRequests.log    # HTTP requests
└── sessionReplay.log      # User interaction events
```

Viewing Logs:

```
# View recent server logs
tail -f .manus-logs/devserver.log

# Search for errors
grep "ERROR" .manus-logs/devserver.log

# View network requests
grep "POST" .manus-logs/networkRequests.log
```

Error Tracking (Sentry):

- All errors are automatically sent to Sentry
- Access Sentry dashboard for error analysis
- Set up alerts for critical errors

8.4 Database Backups

Automated Backups:

- Manus platform automatically backs up PostgreSQL daily
- Backups are retained for 30 days
- Can restore from any backup via Manus UI

Manual Backup:

```
# Export database
pg_dump postgresql://user:password@host/aglc_crm > backup.sql

# Restore database
psql postgresql://user:password@host/aglc_crm < backup.sql
```

---

9. Troubleshooting Guide

9.1 Common Issues & Solutions

Issue: Application Won't Start

Symptoms: Server crashes on startup, "Cannot connect to database"

Diagnosis:

```
# Check database connection
psql $DATABASE_URL -c "SELECT 1"

# Check environment variables
echo $DATABASE_URL
echo $OPENAI_API_KEY

# Check logs
tail -f .manus-logs/devserver.log
```

Solutions:

1. Verify DATABASE_URL is correct
2. Ensure PostgreSQL is running
3. Check network connectivity to database
4. Verify database credentials

Issue: Tasks Not Being Assigned

Symptoms: New tasks created but not assigned to team members

Diagnosis:

```
# Check task assignment logic
SELECT * FROM tasks WHERE assignedTo IS NULL AND status = 'Pending';

# Check if Planner Agent ran
SELECT * FROM agentLogs WHERE agentName = 'planner' ORDER BY timestamp DESC
LIMIT 10;
```

Solutions:

1. Verify Planner Agent is running
2. Check agent logs for errors
3. Manually assign tasks if agent fails
4. Review task assignment logic

Issue: WhatsApp Messages Not Received

Symptoms: Client messages sent to WhatsApp but not appearing in system

Diagnosis:

```
# Check Twilio webhook logs
# Check message routing logic
SELECT * FROM communications WHERE type = 'Message' ORDER BY timestamp DESC;

# Check for errors in agent logs
SELECT * FROM agentLogs WHERE agentName = 'whatsapp' ORDER BY timestamp
DESC;
```

Solutions:

1. Verify Twilio webhook URL is correct
2. Check Twilio credentials
3. Test webhook with Postman
4. Review message routing logic

Issue: Slow Dashboard Loading

Symptoms: Executive dashboard takes >5 seconds to load

Diagnosis:

```
# Check slow queries
EXPLAIN ANALYZE SELECT * FROM cases WHERE status = 'Active';

# Check database connection pool
SELECT count(*) FROM pg_stat_activity;

# Check indexes
SELECT * FROM pg_indexes WHERE tablename = 'cases';
```

Solutions:

1. Add missing indexes
2. Optimize slow queries
3. Increase database connection pool
4. Implement caching with Redis
5. Break large queries into smaller ones

9.2 Performance Troubleshooting

Check Application Performance:

```
# Monitor CPU usage
top -p $(pgrep -f "node.*index.ts")

# Monitor memory usage
ps aux | grep node

# Check network latency
ping database-host

# Check database query performance
SELECT query, mean_time FROM pg_stat_statements ORDER BY mean_time DESC
LIMIT 10;
```

Optimization Steps:

1. Identify slow queries using `EXPLAIN ANALYZE`
2. Add indexes to frequently queried columns
3. Implement caching for read-heavy operations
4. Break large operations into smaller chunks
5. Use connection pooling for database

9.3 Security Troubleshooting

Check for Security Issues:

```
# Verify SSL/TLS is enabled
curl -I https://your-domain.com

# Check for exposed secrets
grep -r "sk-" server/ client/  # Should not find any

# Review audit logs
SELECT * FROM auditLogs WHERE action = 'DELETE' ORDER BY timestamp DESC;

# Check for unauthorized access
SELECT * FROM users WHERE role = 'admin' AND lastSignedIn < NOW() - INTERVAL
30 DAY;
```

---

10. Maintenance & Support

10.1 Regular Maintenance Tasks

Daily:

- Monitor error logs
- Check system health
- Verify backups completed

Weekly:

- Review performance metrics
- Check for security updates
- Analyze user feedback

Monthly:

- Database optimization
- Dependency updates
- Security audit
- Performance review

Quarterly:

- Full security audit
- Disaster recovery drill
- Capacity planning review
- Team training

10.2 Dependency Management

Check for Updates:

```
pnpm outdated
```

Update Dependencies:

```
# Update all dependencies
pnpm update

# Update specific package
pnpm update @trpc/server

# Check for security vulnerabilities
pnpm audit
```

Update Node.js:

```
# Check current version
node --version

# Update to latest LTS
nvm install 22
nvm use 22
```

10.3 Support & Escalation

Support Channels:

1. **Internal:** Slack #aglc-crm-support
2. **External:** Email support@aglc.com
3. **Emergency:** Call +966-XX-XXXX-XXXX

Escalation Path:

1. Level 1: Support team (troubleshooting, basic fixes)
2. Level 2: Development team (code issues, complex problems)
3. Level 3: Architecture team (system design, major changes)
4. Level 4: Executive team (business decisions, budget)

SLA (Service Level Agreement):

- Critical (system down): 1 hour response, 4 hours resolution

- High (major feature broken): 4 hours response, 24 hours resolution
- Medium (minor issue): 24 hours response, 3 days resolution
- Low (enhancement request): 1 week response, 2 weeks resolution

---

11. Security & Compliance

11.1 Data Security

Encryption at Rest:

```
// All sensitive data is encrypted in database
import crypto from 'crypto';

function encryptData(data: string, key: string): string {
  const cipher = crypto.createCipher('aes-256-cbc', key);
  let encrypted = cipher.update(data, 'utf8', 'hex');
  encrypted += cipher.final('hex');
  return encrypted;
}

function decryptData(encrypted: string, key: string): string {
  const decipher = crypto.createDecipher('aes-256-cbc', key);
  let decrypted = decipher.update(encrypted, 'hex', 'utf8');
  decrypted += decipher.final('utf8');
  return decrypted;
}
```

Encryption in Transit:

- All API calls use HTTPS/TLS
- All database connections use SSL
- All external API calls use HTTPS

Key Management:

- Encryption keys stored in environment variables
- Keys rotated quarterly

- Keys never logged or exposed

11.2 Access Control

Role-Based Access Control (RBAC):

```
enum UserRole {
  USER = 'user',      // Can only see own tasks/cases
  ADMIN = 'admin',    // Can see all data, manage users
}

// In tRPC procedures
const adminProcedure = protectedProcedure.use(({ ctx, next }) => {
  if (ctx.user.role !== 'admin') {
    throw new TRPCError({ code: 'FORBIDDEN' });
  }
  return next({ ctx });
});
```

Data Access Rules:

- Users can only see their own tasks and assigned cases
- Admins can see all data
- Clients can only see their own cases
- Audit logs are read-only

11.3 Compliance

Attorney-Client Privilege:

- All communications are encrypted
- Access is restricted to authorized personnel
- Communications are never shared without consent
- Audit logs track all access

Data Privacy (GDPR, CCPA):

- Users can request data export

- Users can request data deletion
- Data retention policy: 7 years for legal, 1 year for others
- Privacy policy published on website

Compliance Checklist:

- [ ] Data encryption enabled
- [ ] Access control implemented
- [ ] Audit logs enabled
- [ ] Backups tested
- [ ] Disaster recovery plan documented
- [ ] Security audit completed
- [ ] Privacy policy published
- [ ] Terms of service published
- [ ] Cyber insurance in place

---

12. Scaling & Performance

12.1 Scaling Strategy

Vertical Scaling (Increase Resources):

- Increase server CPU/RAM
- Increase database resources
- Increase file storage capacity

Horizontal Scaling (Add Servers):

- Run multiple backend instances behind load balancer
- Use read replicas for database
- Use CDN for static assets

Caching Strategy:

- Cache frequently accessed data (clients, case templates)
- Use Redis for session storage
- Implement query result caching

12.2 Performance Optimization

Database Optimization:

```sql
-- Add indexes for frequently queried columns
CREATE INDEX idx_cases_clientId ON cases(clientId);
CREATE INDEX idx_tasks_assignedTo ON tasks(assignedTo);
CREATE INDEX idx_tasks_deadline ON tasks(deadline);

-- Analyze query performance
EXPLAIN ANALYZE SELECT * FROM cases WHERE status = 'Active';

-- Vacuum and analyze
VACUUM ANALYZE;
```

API Optimization:

- Implement pagination for large result sets
- Use field selection (only fetch needed columns)
- Implement request batching
- Use compression (gzip)

Frontend Optimization:

- Code splitting and lazy loading
- Image optimization and lazy loading
- Minimize bundle size
- Use CDN for static assets

12.3 Monitoring & Metrics

Key Performance Indicators (KPIs):

- API response time: <200ms (p95)
- Database query time: <100ms (p95)
- Page load time: <3s (p95)
- Uptime: >99.9%
- Error rate: <0.1%

Monitoring Tools:

- Application Performance Monitoring (APM): Sentry
- Database Monitoring: PostgreSQL logs
- Infrastructure Monitoring: Manus platform
- User Monitoring: Analytics

Conclusion

This knowledge transfer document provides a comprehensive guide to understanding, maintaining, and extending the AGLC Law Firm CRM system. Key points to remember:

1. **Architecture:** Five-layer system with frontend, backend, database, and external services
2. **Technology:** React 19, Express 4, PostgreSQL, tRPC, OpenAI
3. **Development:** Type-safe, modular, well-tested code
4. **Operations:** Automated deployment, monitoring, and backups
5. **Security:** Encryption, access control, audit logs, compliance
6. **Scaling:** Horizontal and vertical scaling strategies

For questions or issues, refer to the troubleshooting guide or contact the development team.

Document Prepared By: Manus AI
Date: March 11, 2026
Version: 1.0

Last Updated: March 11, 2026
Next Review: June 11, 2026